A Cloud-Edge Microservice Scheduler Based on Trace-Guided Adjustment Reasoning

Yuxin Zhao
Toyohashi University of Technology
Toyohashi, Japan
zhao.yuxin.wt@tut.jp

Abstract

Microservice placement in cloud-edge environments often faces inefficiencies caused by the gap between past scheduling decisions and actual performance observations. This paper proposes a trace-guided adjustment reasoning framework for cloud-edge microservice scheduling. The framework transforms runtime traces into prior knowledge for deployment adjustment through three stages: state inference, suggestion generation, and saturation-aware scoring. These stages infer service dependencies and contention risk, generate candidate placements, and rank them according to expected benefit and resource pressure. We evaluate the framework on a resource-constrained Kubernetes testbed and compare it with a baseline adjustment strategy. The results show that the proposed method improves execution stability and reduces high-percentile latency without increasing the total resource footprint. These results suggest that trace-derived bottleneck information can support more stable microservice placement in compute-limited cloud-edge environments.

Keywords

Microservices, Kubernetes, Cloud-edge computing, Trace-driven scheduling, Resource reconfiguration

1 Introduction

Modern microservice scheduling needs more than feasible Pod placement in cloud-edge environments. Edge nodes may differ in capacity and network conditions, which makes placement decisions sensitive to where services are deployed [4]. The Kubernetes scheduler can filter feasible nodes and score candidates, but this mechanism mainly decides where a Pod can run [8]. Feasible placement may still suffer from runtime contention or CPU throttling, which can slow down service execution [11]. Therefore, scheduling support should consider how service delays propagate through the call chain, not only whether each Pod can be placed.

1.1 Related Work

These challenges have led to studies that improve microservice management at different stages. This subsection reviews these studies and explains why trace-derived dependency and bottleneck information is still not fully used for scheduling support.

Placement and scheduling.

Placement and scheduling methods improve microservice performance by selecting suitable nodes under resource and network constraints. Pallewatta et al. [10] show that node heterogeneity and resource limits strongly affect microservice placement in fog environments. MOTAS [9] further uses topology information and distributed trace analysis, but its focus remains on improving node selection before runtime bottlenecks are fully exposed. Therefore, an initially efficient placement may become unsuitable during execution, so scheduling support should use runtime evidence to decide which services need later adjustment.

Autoscaling and reconfiguration.

Auto-scaling methods improve runtime adaptation by adjusting resources or replicas after workload changes are observed. Smart HPA [1] shows a recent direction that improves horizontal pod autoscaling by considering resource efficiency in microservice architectures. However, Chamulteon [2] shows that independent scaling may cause bottleneck shifting or oscillation when the bottleneck moves to another dependent service. Therefore, reconfiguration should use evidence about service interactions before deciding which service should be adjusted.

Trace-based diagnosis.

Trace-based root cause analysis (RCA) identifies services or resources responsible for observed degradation. Sage [5] uses RPC-level traces, causal modeling, and counterfactual reasoning to locate QoS root causes and connect diagnosis with corrective actions. However, RCA-based correction is usually triggered after degradation occurs, and may focus on the diagnosed component while missing alternative reconfiguration choices, such as redistributing resources from less critical services [1]. Therefore, trace evidence should be used not only to explain an observed problem, but also to guide adjustment candidates before the next reconfiguration decision.

Learning-based management.

Learning-based methods learn resource-performance relationships and use them to select management actions. Sinan [14]

predicts end-to-end tail latency and QoS violation risk under different per-tier resource allocations. CoScal [12] applies reinforcement learning to combine scaling and brownout decisions. These methods can automate resource management, but they depend on training data, predefined action spaces, and feedback quality. Therefore, trace-driven adjustment reasoning is needed to infer adjustment effects before later candidates are selected.

1.2 Research Gap and Positioning

Figure 1 shows the position of the proposed method in the microservice management workflow.

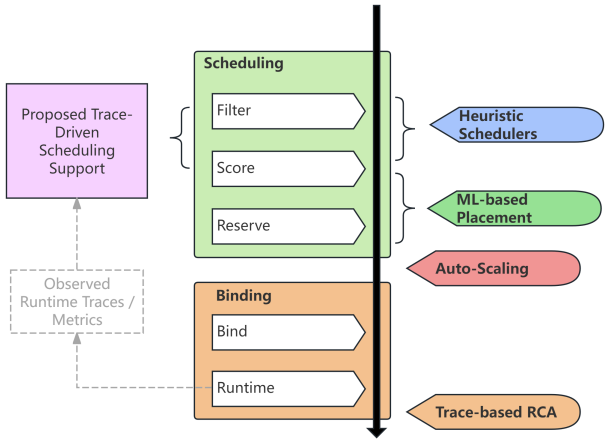

Figure 1: Positioning of the proposed trace-driven scheduling support.

The main innovation of this work is *trace-driven adjustment reasoning for scheduling support*. Existing methods usually improve a specific decision stage, while this work focuses on the reasoning process between an observed trace and a later adjustment decision. This distinction matters because an adjustment can change the system state itself, and subsequent decisions must account for this changed state. Therefore, the proposed method uses runtime traces as evidence to refine the following scheduling-support logic.

This innovation is supported by three design points:

- **Trace-based priority identification.** The method extracts dependency and bottleneck-related hints from observed call chains, such as frequently invoked services, repeated calls, fan-out structures, and latency-sensitive paths.
- **Effect-oriented candidate generation.** The method converts trace evidence into adjustment candidates, such as resource redistribution, replica adjustment, placement change, or coordinated adjustment with dependent services.
- **Recursive impact-aware feedback.** Each candidate is evaluated by its expected benefit and risk, including latency reduction, resource pressure, contention, or bottleneck shifting. The estimated effect is then used to refine subsequent adjustment logic.

2 Proposed model concepts

2.1 System Architecture

Figure 2 shows how the proposed method is integrated with the Kubernetes workflow. The standard control plane still manages cluster state and scheduling decisions through the API server, scheduler, controller manager, and etcd. The proposed trace-guided scheduler is added as a support layer rather than a replacement of these components. Therefore, the architecture keeps compatibility with Kubernetes while allowing runtime evidence to influence later scheduling and reconfiguration decisions.

The method uses runtime traces and metrics to support adjustment reasoning. Metrics from Node Exporter and Prometheus show resource pressure on nodes and containers, while traces from Istio sidecars and Jaeger show how requests move across services. By combining these two sources, the trace-guided scheduler can identify services and paths that should be considered for adjustment, then estimate the benefit and risk of each candidate. The result is passed back through the Kubernetes workflow and provides the basis for the trace-guided optimization algorithm described next.

2.2 Trace-Guided Population Generation

GA-style search is suitable for scheduling because placement and resource adjustment form a combinatorial search problem. Recent studies show that evolutionary algorithms are still used for scheduling optimization [13], and that heuristic initialization or knowledge-based mutation can guide the search toward better candidates [7]. This means GA design does not need to rely only on parameter tuning or random mutation; useful evidence can also shape how new populations are generated. Therefore, this work uses runtime traces to generate candidates from observed execution behavior rather than arbitrary parameter changes.

Figure 3 illustrates the trace-guided population generation process. When a trace contains a severe delay, as shown by the red segment, all services and nodes on the corresponding call chain are collected as the adjustment scope. This scope is not treated as a final solution, but as a baseline group for generating new candidates. The next population is then constructed by recombining this group with alternative mappings observed in other traces, and the resulting candidates are evaluated by the fitness function described next.

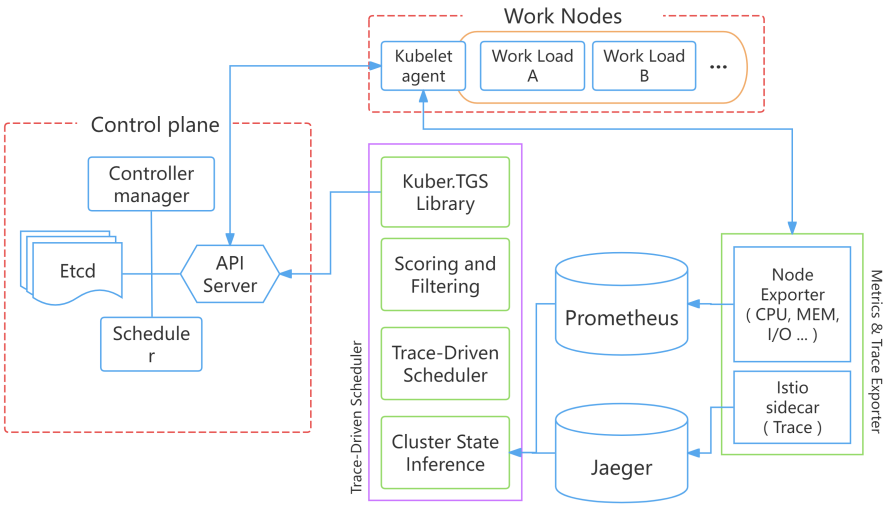

Figure 2: Architecture of the proposed trace-guided scheduling support method.

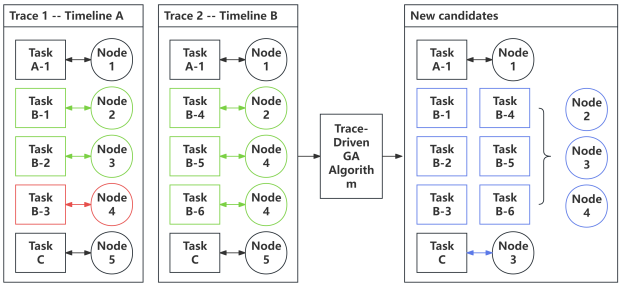

Figure 3: Trace-guided population generation.

2.3 Fitness Evaluation

The generated candidates are evaluated by a trace-guided fitness function. In microservice systems, local resource adjustment may shift the bottleneck to dependent services, so a candidate should be evaluated by both its expected improvement and its possible risk [2]. Therefore, the fitness function does not aim to prove global optimality, but to compare adjustment candidates under trace-derived service importance and current resource pressure. We define the fitness as follows:

$$\begin{aligned} Fit_t(c) &= B_t(c) - P_t(c), \\ B_t(c) &= \sum_{s \in S_c} I_s^t H_s^t G_s(c), \\ P_t(c) &= \sum_{n \in N_c} \Delta\Psi_n(c) + \sum_{s \in S_c} \eta_s R_s(c). \end{aligned} \tag{1}$$

Here, S_c and N_c denote the services and nodes affected by candidate c. The benefit term rewards adjustments to trace-important and bottleneck-related services, while the penalty term reduces the fitness when the candidate increases node pressure or expands resources unnecessarily.

The gain and penalty terms depend on the adjustment type. For example, when c increases CPU allocation, the reward and node-pressure penalty are adjusted as follows:

$$\begin{aligned} G_s^{cpu}(c) &= 1 - e^{-\alpha \Delta CPU_s}, \\ \Delta\Psi_n(c) &= \Psi(U_n'(c)) - \Psi(U_n), \\ \Psi(U_n) &= \max(0, U_n - \theta)^2. \end{aligned} \tag{2}$$

The first term gives diminishing reward to repeated CPU expansion. The second term penalizes candidates that push an already loaded node beyond the pressure threshold θ. In this way, the fitness function favors adjustments that are supported by trace evidence while suppressing candidates that may cause unnecessary expansion or new contention.

3 Methodology

This section describes the implementation and evaluation setup of the proposed method. It first summarizes the implementation workflow with pseudocode, showing how runtime observations are converted into scheduling-support strategies. It then presents the cluster setting and workload setting used in the evaluation. Together, these subsections clarify how the proposed method is implemented, executed, and evaluated in a resource-constrained Kubernetes environment.

3.1 Implementation Workflow

In this prototype, we focus on trace-guided candidate filtering rather than a full GA population update. Runtime traces are used to infer service importance and bottleneck tendency, while cluster metrics are used to estimate node pressure. These inferred states are then used by the fitness function to evaluate candidate adjustment strategies. This design allows us to examine whether the proposed scoring logic can retain useful trace-driven strategies under resource-constrained conditions.

Algorithm 1 summarizes the implemented candidate filtering process. The prototype first infers service state from traces and node state from metrics, then evaluates each candidate adjustment with a trace-guided fitness function. Candidates with sufficient fitness are retained as scheduling-support strategies. This workflow is used to verify whether the proposed scoring logic can select trace-driven adjustments that improve execution stability under resource-constrained conditions.

Algorithm 1 Trace-Guided Candidate Filtering

Require: Traces T, metrics M, candidate set C, threshold τ
Ensure: Retained strategy set $\mathcal{R}$

$SvcState \leftarrow \textsc{InferServiceState}(T)$
$NodeState \leftarrow \textsc{InferNodeState}(M)$
$\mathcal{R} \leftarrow \emptyset$
function $\textsc{FilterCandidates}(C, SvcState, NodeState)$
  for all $c \in C$ **do**
    $S_c \leftarrow \textsc{AffectedServices}(c)$
    $N_c \leftarrow \textsc{AffectedNodes}(c)$
    $B \leftarrow \sum_{s \in S_c} I_s H_s G_s(c)$
    $P \leftarrow \sum_{n \in N_c} \Delta\Psi_n(c) + \sum_{s \in S_c} \eta_s R_s(c)$
    $fitness \leftarrow B - P$
    if $fitness \geq \tau$ **then**
      $\mathcal{R} \leftarrow \mathcal{R} \cup \{(c, fitness)\}$
    end if
  end for
  return $\textsc{SortByFitness}(\mathcal{R})$
end function
return $\textsc{FilterCandidates}(C, SvcState, NodeState)$

3.2 Cluster Environment

The experiment is conducted on a single-node minikube cluster. The cluster provides 8 CPU cores and 16 GB of memory. Since all microservice replicas run on the same node, this setting creates a constrained shared-resource environment and allows us to focus on resource and replica reconfiguration. Table 11-a summarizes the cluster environment.

3.3 Application and Workload

This study adopts μBench as the base benchmark to build a controllable workload for evaluating scheduling effects. DeathStarBench is a widely used microservice benchmark suite for realistic end-to-end cloud and edge applications [6]. However, such realism makes it harder to isolate the effect of service workload size and call-chain structure. Therefore, this study uses μBench, which provides a configurable benchmark factory for microservice applications [3], to replay simplified Alibaba-derived traces with CPU-bound π computation tasks.

Table 1: Experimental settings.

1-a: Cluster environment.

Item	Setting
Cluster	Single-node minikube
CPU	8 cores
Memory	16 GB
K8s client	v1.30.0
K8s server	v1.29.2
Kustomize	v5.0.4

1-b: Workload scale.

Item	Setting
Benchmark	μBench
Trace source	Alibaba-derived
Services	12
Entry	$s0$
Request types	3
Requests/type	20
Total requests	60
Concurrency	6
Timeout	5 min

The test application consists of 12 microservices, where $s0$ acts as the entry service. The workload preserves the trace-based call-chain structure, while the computation size of selected services is adjusted to create different service costs. The test data contain three request types, and each type is sent 20 times. Therefore, each run sends 60 requests in total, with a fixed concurrency level of 6 and a request timeout of 5 minutes. The overall workload scale is summarized in Table 11-b.

3.4 Deployment Strategies

The study compare three deployment strategies under the same application and workload setting. The comparison focuses on how each strategy arranges replicas and resource requests under limited cluster resources.

- **Baseline.** Each of the 12 services has two replicas, resulting in 24 workload pods. No service-specific resource request is actively tuned.
- **Proposed.** The total number of workload pods remains 24. Replica allocation is adjusted according to trace-derived hints: replicas of less critical services are reduced, while important or bottleneck-related services receive higher resource requests.
- **RCA-based.** Resource requests are increased for compute-heavy services, and replicas are added for hotspot services to avoid making them new bottlenecks. As a result, the total number of workload pods increases to about 30.

4 Experiment Results

Table 2 summarizes the end-to-end latency results. Under the same resource-constrained testbed, the proposed trace-driven strategy achieves the lowest $p50$, $p95$, and standard deviation among the three strategies. This suggests that the main benefit of the proposed method is more stable tail-latency behavior and lower request-to-request variation, rather than a simple reduction in average latency.

Table 2: Latency comparison of the three strategies.

Strategy	Mean	$p50$	$p95$	Std. dev.	Max
Proposed	130.396	124.456	146.870	11.344	159.685
Baseline	129.850	128.593	151.190	12.544	161.559
RCA-based	131.053	128.743	160.324	16.610	184.440

4.1 Analysis

Figure 4 shows that the proposed strategy produces a more stable latency distribution than the other two strategies. Its median and upper percentile range are lower, and the individual request points are more tightly distributed. This indicates that the trace-driven strategy improves not only tail latency but also request-to-request stability, because it reduces extreme long-tail behavior rather than only shifting the average response time. The remaining question is whether this improvement comes from better reconfiguration or simply from higher resource consumption.

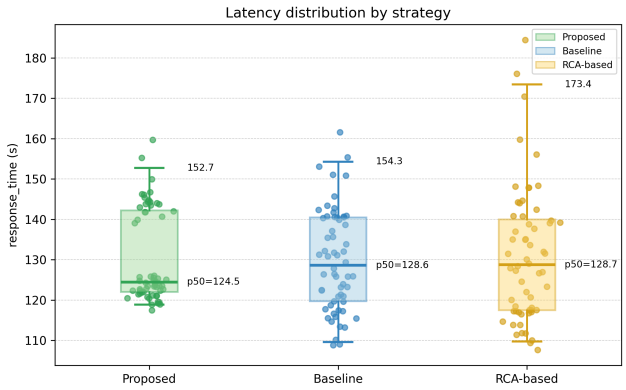

Figure 4: Request latency distribution.

Figure 5 shows that the proposed strategy improves latency stability without relying on stronger CPU resource consumption. During the rising phase, the CPU utilization of the three strategies increases in a similar range, and their utilization levels remain close during the main execution period. This indicates that no strategy obtains a clear advantage from additional CPU resource tilt; instead, the proposed strategy shows a smoother utilization curve while achieving better latency distribution. Therefore, the improvement is more likely related to trace-guided resource and replica reconfiguration, rather than simple resource stacking.

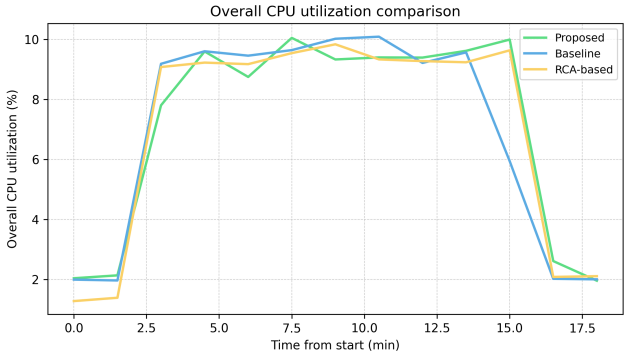

Figure 5: Overall CPU utilization comparison.

From these results, we can see that the proposed strategy improves stability without relying on resource stacking. The RCA-based strategy is allowed to increase replicas for hotspot services and allocate more resources to compute-heavy services, so it should have more room for resource expansion than the proposed strategy. However, its tail latency and request dispersion are worse, which suggests that local resource expansion under shared-resource constraints may increase contention or shift the bottleneck to dependent services [2]. This supports the design of the proposed fitness function, which evaluates not only the expected benefit of an adjustment but also its resource-side risk.

4.2 Discussion and Future work

This study demonstrates that trace-driven adjustment reasoning can improve tail-latency stability under limited resources, but the current validation is still preliminary. The experiments are conducted on a single-node minikube cluster, so inter-node placement and network-aware scheduling remain future work. In addition, practical deployment requires update control, because frequent reconfiguration may introduce overhead or unstable feedback. The method also depends on observed traces, so unseen call chains or sudden workload shifts may reduce the accuracy of trace-derived hints. Future work will therefore extend the framework to multi-node environments and study adaptive update policies with cooldown intervals, update thresholds, and fallback mechanisms.

References

[1] Hussain Ahmad, Christoph Treude, Markus Wagner, and Claudia Szabo. 2024. Smart HPA: A Resource-Efficient Horizontal Pod Auto-Scaler for Microservice Architectures. In *Proceedings of the IEEE 21st*

International Conference on Software Architecture (ICSA '24). IEEE, 46–57. doi:10.1109/ICSA59870.2024.00013

[2] André Bauer, Veronika Lesch, Laurens Versluis, Alexey Ilyushkin, Nikolas Herbst, and Samuel Kounev. 2019. Chamulteon: Coordinated Auto-Scaling of Micro-Services. In *Proceedings of the 2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS '19)*. IEEE, 2015–2025. doi:10.1109/ICDCS.2019.00199

[3] Andrea Detti, Ludovico Funari, and Luca Petrucci. 2023. μBench: An Open-Source Factory of Benchmark Microservice Applications. *IEEE Transactions on Parallel and Distributed Systems* 34, 3 (2023), 968–980. doi:10.1109/TPDS.2023.3236447

[4] Alexandre Peixoto Ferreira and Jon Hermes. 2024. Adapting Kubernetes for High-Performance IoT Edge Deployments. https://developer.arm.com/community/arm-community-blogs/b/embedded-and-microcontrollers-blog/posts/adapting-kubernetes-high-performance-iot-edge-deployments. Accessed: 2026-05-14.

[5] Yu Gan, Mingyu Liang, Sundar Dev, David Lo, and Christina Delimitrou. 2021. Sage: Practical and Scalable ML-Driven Performance Debugging in Microservices. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '21)*. Association for Computing Machinery, New York, NY, USA, 135–151. doi:10.1145/3445814.3446700

[6] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, Kelvin Hu, Meghna Pancholi, Yuan He, Brett Clancy, Chris Colen, Fukang Wen, Catherine Leung, Siyuan Wang, Leon Zaruvinsky, Mateo Espinosa, Rick Lin, Zhongling Liu, Jake Padilla, and Christina Delimitrou. 2019. An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud and Edge Systems. In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19)*. Association for Computing Machinery, New York, NY, USA, 3–18. doi:10.1145/3297858.3304013

[7] Cheng Gong, Yang Nan, Lie Meng Pang, Hisao Ishibuchi, and Qingfu Zhang. 2024. Heuristic Initialization and Knowledge-Based Mutation for Large-Scale Multi-Objective 0-1 Knapsack Problems. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO '24)*. Association for Computing Machinery, 178–186. doi:10.1145/3638529.3654006

[8] Kubernetes. 2024. Kubernetes Scheduler. https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/. Accessed: 2026-05-14.

[9] Xin Li, Junsong Zhou, Xin Wei, Dawei Li, Zhuzhong Qian, Jie Wu, Xiaolin Qin, and Sanglu Lu. 2023. Topology-Aware Scheduling Framework for Microservice Applications in Cloud. *IEEE Transactions on Parallel and Distributed Systems* 34, 5 (2023), 1635–1649. doi:10.1109/TPDS.2023.3238751

[10] Samodha Pallewatta, Vassilis Kostakos, and Rajkumar Buyya. 2019. Microservices-Based IoT Application Placement within Heterogeneous and Resource Constrained Fog Computing Environments. In *Proceedings of the 12th IEEE/ACM International Conference on Utility and Cloud Computing (UCC '19)*. Association for Computing Machinery, New York, NY, USA, 71–81. doi:10.1145/3344341.3368800

[11] Ara Pulido. 2023. Kubernetes CPU Limits and Requests: A Deep Dive. https://www.datadoghq.com/blog/kubernetes-cpu-requests-limits/. Accessed: 2026-05-14.

[12] Minxian Xu, Chenghao Song, Shashikant Ilager, Sukhpal Singh Gill, Juanjuan Zhao, Kejiang Ye, and Cheng-Zhong Xu. 2022. CoScal: Multifaceted Scaling of Microservices With Reinforcement Learning. *IEEE Transactions on Network and Service Management* 19, 4 (2022), 3995–4009. doi:10.1109/TNSM.2022.3210211

[13] Wenqiang Zhang, Guanwei Xiao, Mitsuo Gen, Huili Geng, Xiaomeng Wang, Miaolei Deng, and Guohui Zhang. 2024. Enhancing Multi-Objective Evolutionary Algorithms with Machine Learning for Scheduling Problems: Recent Advances and Survey. *Frontiers in Industrial Engineering* 2 (2024). doi:10.3389/fieng.2024.1337174

[14] Yanqi Zhang, Weizhe Hua, Zhuangzhuang Zhou, G. Edward Suh, and Christina Delimitrou. 2021. Sinan: ML-Based and QoS-Aware Resource Management for Cloud Microservices. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '21)*. Association for Computing Machinery, New York, NY, USA, 167–181. doi:10.1145/3445814.3446693